Depth-First Search

Depth-First Search (DFS) is a graph traversal algorithm that explores as far as possible along each branch before backtracking.

Natural Explanation

Imagine exploring a maze: you follow one path deep into the maze, and when you hit a dead end, you backtrack to the last junction and try another path. DFS is like exploring exhaustively in one direction before switching direction.

Formal Definition

Given a graph $G = (V, E)$ and a starting vertex s:

DFS systematically explores all vertices reachable from s by recursively visiting unvisited neighbors. It uses implicit (recursive call stack) or explicit [[Stacks|stack]] storage.

Pseudocode (Recursive)

```
DFS(graph, start_vertex)
    visited = empty set
    DFS-VISIT(start_vertex, visited)

DFS-VISIT(vertex, visited)
    add vertex to visited
    for each neighbor in graph.NEIGHBORS(vertex):
        if neighbor not in visited:
            DFS-VISIT(neighbor, visited)
```

Pseudocode (Iterative with Stack)

```
DFS-ITER(graph, start_vertex)
    visited = empty set
    stack = empty stack
    stack.PUSH(start_vertex)

    while not stack.ISEMPTY():
        vertex = stack.POP()
        if vertex not in visited:
```

```
add vertex to visited
for each neighbor in graph.NEIGHBORS(vertex):
    if neighbor not in visited:
        stack.PUSH(neighbor)
```

Complexity

- **Time**: $O(|V| + |E|)$ — visit each vertex once, examine each edge once
- **Space**: $O(|V|)$ — stack depth at most $|V|$ in worst case (very deep or long path)

Key Properties

Discovers connected components: run DFS from each unvisited vertex to partition vertices into components.

Creates a DFS tree (or forest): the set of edges used to reach vertices forms a tree rooted at start vertex.

Visits all reachable vertices: everything connected to the start is visited.

Common Applications

- **Reachability**: can we reach vertex v from vertex u?
- [[**Topological Sorting**]]: for directed acyclic graphs
- [[**Strongly Connected Components**]]: finding all strongly connected components
- **Cycle detection**: a back edge (edge to an ancestor in DFS tree) indicates a cycle
- **Finding bridges/articulation points**: in undirected graphs
- **Path finding**: DFS tree gives one path from root to any reachable vertex

---

Related

- [[Breadth-First Search]]
- [[Graphs]]
- [[Stacks]]
- [[Tree Traversals]]
- [[Topological Sorting]]
- [[Strongly Connected Components]]